INT161 · Basic Backend Development

HTTP Basics

Node event loop, the http module, and reading the method, URL, query and path from a request.

Node.js Architecture | Core Modules, CJS and ESM | The http Module | HTTP Methods

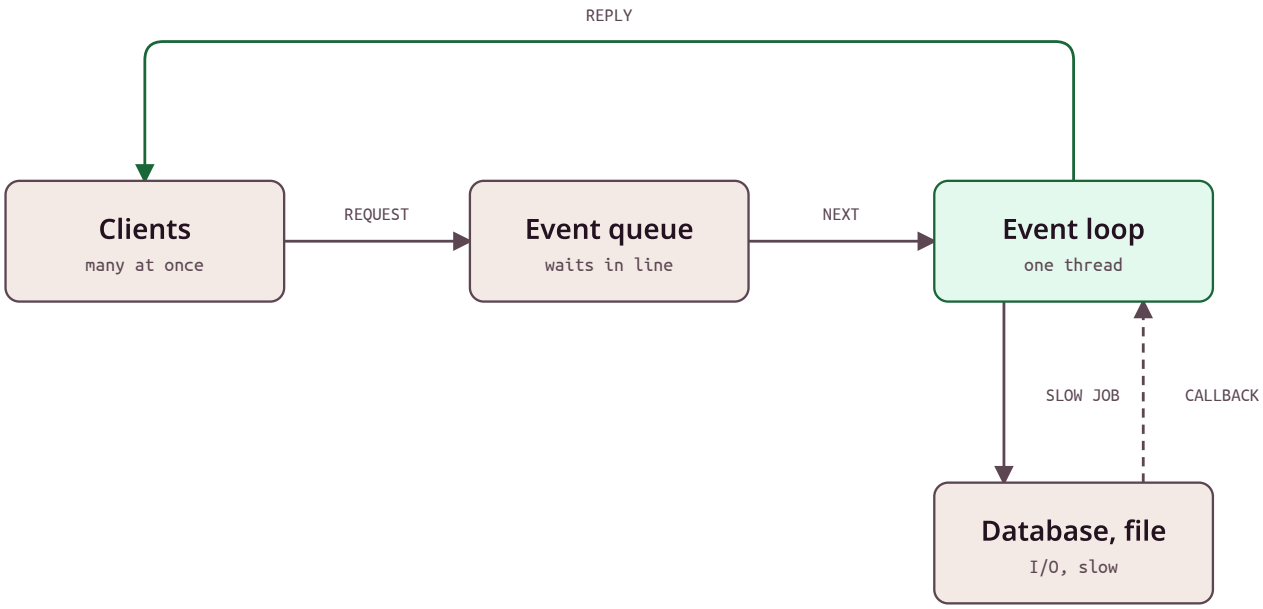
Query and Path Parameters | Exercise: Multiplication Table | Glossary

01

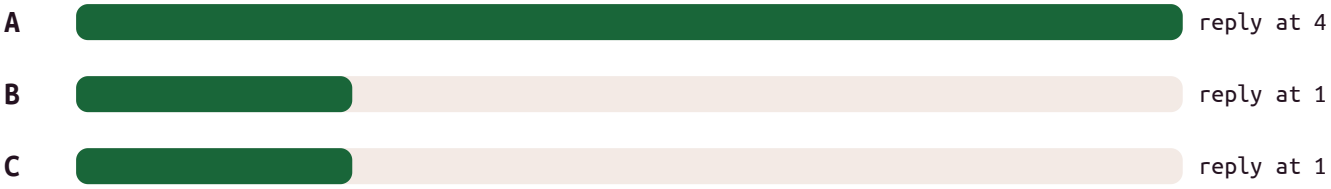
Node.js Architecture

Goal of the class: create an **HTTP server**, handle the **request** and **response** objects, and read the **request parameters** and path.

- The input of your program is an **HTTP** request. The output is an HTTP response. **Node** turns both into objects so you do not parse text by hand.



One by one | Event queue



Reply order: B, C, A

A = Needs the database · B = Static page · C = Static page

A slow **request** does not hold the others. Whoever finishes first is answered first.

02

Core Modules, CJS and ESM

The lecturer compared **built-in modules** to a condo that comes with built-in furniture. They arrive with Node, and you cannot remove them.

- Built-in is not the same as ready to use. You still import a **module** before you use it.
- Modules that are not built in are installed with **npm**.
- Common ones: fs for files, http for **servers**, path, os, events, util, **stream**, crypto, url, querystring.
- This class focuses on http, because it is the HTTP server that talks to the **front end**.

ESM (use this)

```
import * as http from 'node:http'
```

The official standard. Parsed up front, loads modules at the same time, and allows **tree shaking**.

CommonJS (legacy)

```
const http = require('node:http')
```

Older style. Loads one file after another. Many web examples still use it.

Why ESM here

The lecturer asked several AI tools which to use. Every one said ESM for a new project. Put imports at the top of the file. Examples you find online may use require, so convert them.

03

The http Module

The module has four main classes. Two of them Node creates for you on every request.

`http.Server`

The server itself.

listen(**port**, **callback**) starts it. close() stops new **connections**. The request event fires for every request.

`http.IncomingMessage`

The request.

Node builds it for you. Read request.method, request.url and request.**headers**.

`http.ServerResponse`

The response.

Node builds it too. writeHead() sets the status and headers, write() adds **body**, end() finishes and sends.

`http.ClientRequest`

A request you send.

Used when your **back end** calls another server, such as a courier **API**. Then you are the **client**.

http-server.js

```
import * as http from 'node:http';
const listener = function (request, response) {
  const { url, method } = request;
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  response.end(`You made a ${method} request to ${url}`);
};
const server = http.createServer(listener);
server.listen(3000, () => {
  console.log('Server running at http://127.0.0.1:3000/');
});
server.on((req, res) => {
  console.log('Request received:', req.method, req.url);
});
```

- The handler gets two arguments, and the order is fixed: request first, response second. The names are yours. Swap the names and the code breaks.
- createServer only describes what to do. Nothing happens until listen. The default port in Node examples is 3000.
- The listen callback is how you know it started. Without it the server is silent.
- end() finishes the response and sends it. write() can be called many times first.
- With Content-Type text/plain a **browser** prints your HTML as text instead of rendering it.

two terminals

```
# terminal 1
$ node http-server.js
Server running at http://127.0.0.1:3000/
# terminal 2: -i shows the response headers
$ curl -i http://localhost:3000/user
HTTP/1.1 200 OK
Content-Type: text/plain

You made a GET request to /user
```

Port trap

Two servers cannot share a port. If node is still running and you start bun on the same port, bun starts without an error and still receives nothing. Requests keep going to the old node process, so you **debug** code that never runs. Stop the old one first.

04

HTTP Methods

Every request carries a method and a **URL**. Read them with request.method and request.url.

GET

Read data

`/subjects, /students`

POST

Create a new resource

`/subjects (with data)`

PUT

Update existing data

`/subjects/{id} (with data)`

DELETE

Remove a resource

`/subjects/{id}`

- A browser address bar only sends GET. To test POST or PUT, use **curl** or **Postman**.
- The method matters later. In a **REST** API, reading data must use GET and creating must use POST. It shapes how you design **endpoints**.

curl with a method

```
$ curl -i -X PUT http://localhost:3000/user/5 -d '{"name": "John Doe", "role": "Admin"}'
You made a PUT request to /user/5
```

05

Query and Path Parameters

Data arrives in the URL in two ways. Pull a URL apart and see the pieces.

/user?name=Somchai&fav_s... | /user/123 | /multiplication-table/7 | /multiplication-table?nu...

/user?name=Somchai&fav_subj=Back-end Dev&fav_subj=Front-end Dev

urlObj.pathname

"/user"

pathname.split('/')

["","user"]

.filter(Boolean)

["user"]

params.getAll('name')

["Somchai"]

params.getAll('fav_subj')

["Back-end Dev","Front-end Dev"]

split on a leading slash gives an empty first piece, so filter(Boolean) drops it. A name that was not sent gives null. Every value is a string, even 7.

Query string: the part after ?, written name=value and joined with &. A Google search does this: the q parameter holds what you typed. The same name can appear twice, like checkboxes. Use getAll for that, and you get an array.

Reading the query string

```
const urlObj = new URL(request.url, `http://${request.headers.host}`);
const params = urlObj.searchParams;
const name = params.get('name');
const subjects = params.getAll('fav_subj');
response.write(`Name = ${name} Favorite Subjects: ${subjects}\n`);
```

Path: split the **pathname** on the slash. The leading slash makes an empty first piece, so filter it out.

Reading the path

```
const urlObj = new URL(request.url, `http://${request.headers.host}`);
const pathSegments = urlObj.pathname.split('/').filter(Boolean);
// /users/123  ->  ['', 'users', '123']  ->  filter(Boolean)  ->  ['users', '123']
```

- Names must match exactly. A misspelled name is simply a different name, and you get null.
- Front end and back end teams should agree on names before they start.
- **Frameworks** do this for you later. Learn it by hand once so you know what they are doing.

06

Exercise: Multiplication Table

The lecturer's task: return a multiplication table. Output can be plain text. Looks do not matter.

- Accept the number two ways: /multiplication-table?number=7 or /multiplication-table/7.
- If both are sent, the path wins.
- If the path is not /multiplication-table, answer status 400 and ignore the body.
- Every input is a string. Convert it before you multiply.

How to practice

Add the extra rules even though the demo only returns 200. Practice is the **pool**, the exam is the sea. Type it yourself, do not paste. If you get stuck, ask an AI what an error means, not to write the whole program. Only check what the spec says, such as the two input styles. Do not over-build.

One working answer, to compare after you have tried it.

multiplication-table.js (one possible answer)

```
import * as http from 'node:http';
const listener = (request, response) => {
  const url = new URL(request.url, `http://${request.headers.host}`);
  const [route, pathNumber] = url.pathname.split('/').filter(Boolean);
  if (route !== 'multiplication-table') {
    response.writeHead(400);
    return response.end();
  }
  const n = Number(pathNumber ?? url.searchParams.get('number'));
  let table = '';
  for (let i = 1; i <= 12; i++) table += `${n} x ${i} = ${n * i}\n`;
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  response.end(table);
};
http.createServer(listener).listen(3000);
```

07

Glossary

API

Application Programming Interface: the doors a program offers to other programs.

back end

The part on the server: it handles data, rules and the database, and answers the front end with JSON.

body

The data a request carries after the headers. For an API it is usually JSON.

browser

The client most people use, such as Chrome, Edge or Safari.

built-in modules

A module that ships inside Node. Use it right away, no install.

callback

A function you hand over to be called later, when a job finishes.

client

The side that asks. Usually a browser, but it can be any program or device.

connections

An open line between your program and the database. You need one before you can send any SQL.

curl

A command line client. It sends any HTTP method, so it can test POST and PUT. Add -i to see the response headers.

debug

Find and fix a bug, usually by stepping through the code and watching the values.

endpoints

One method plus path an API answers, such as GET /api/subjects.

ESM

The modern module system: import and export. Needs "type": "module" in package.json.

Frameworks

A toolkit that makes a job easier but sets the frame you must write inside.

front end

The part users see and touch. It runs in the browser or on the phone.

headers

Extra information about a message, sent as name: value lines, such as Content-Type.

HTTP

HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

HTTP server

Software on a server machine that receives HTTP requests and sends responses.

module

A file of code you load into another file with import.

Node
: A runtime that runs JavaScript outside the browser, so the same language can write the back end.

npm
: Node Package Manager: installs libraries such as express into node_modules.

pathname
: The part of the URL after the host and before the ?, like /user/123.

pool
: A few ready connections kept open and shared. A request borrows one and gives it back, so the database is not flooded.

port
: A number that says which program on the machine should get the message. Web servers use 80 by default.

Postman
: A tool for sending any HTTP method to your API, so you can test it before the front end exists.

Query string
: The part of the URL after ?, like page=2&size=10. Used to filter or page a list.

request
: What the client sends: method, URL, headers and sometimes a body.

request parameters
: A name=value pair after the ? in a URL. Several are joined with &. Every value arrives as a string.

response
: What the server sends back: status code, headers and a body.

REST
: A style for web APIs: resources named by URLs, actions chosen by HTTP method.

servers
: A program (or the machine running it) that waits for requests and answers them.

stream
: Data that arrives in pieces over time instead of all at once.

tree shaking
: A build step that drops the code you imported but never used. ESM allows it, CJS does not.

URL

The address of a resource: protocol, host, port and path, such as http://localhost:3000/user.